Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Dashboard 'The Power Grid' (SCRUM-25)

1. CONTEXTO
Se requiere construir un dashboard interactivo de simulación de valor con temática cyberpunk para presentaciones de preventa. El stack está **explícitamente definido** en la especificación técnica: HTML5 semántico, CSS3 puro y Vanilla JavaScript ES6+. No existe backend, base de datos ni autenticación. Es una aplicación 100% client-side, estática, de propósito demostrativo.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura elegida: Single-File Application (SFA) con módulos ES6 lógicamente separados en archivos planos.

Se opta por una estructura de archivos planos sin bundler (Webpack, Vite, etc.), sin frameworks de componentes (React, Vue) y sin dependencias npm. El punto de entrada es un único `index.html` que orquesta la carga de módulos JS nativos vía `<script type='module'>`. Los estilos se consolidan en un único `styles.css` organizado por secciones con comentarios de bloque.

Separación lógica de responsabilidades (sin capas técnicas vacías):
- `state.js`: Gestor de estado global (globalState + KPI_CONFIG). Fuente única de verdad.
- `kpi-engine.js`: Motor de cálculo de KPIs, función easeOutCubic y animación de números.
- `canvas-renderer.js`: Todo el código Canvas (partículas, conexiones, velocímetro, radar).
- `ui-controller.js`: Manejadores de eventos DOM, toggle de nodos, activación de Overdrive.
- `main.js`: Punto de entrada, inicialización del requestAnimationFrame loop y wiring de módulos.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS / BORING TECHNOLOGY
- **¿Por qué NO microservicios ni backend?** No hay persistencia de datos, autenticación ni lógica de servidor. Agregar un backend sería BDUF puro sin ningún criterio de aceptación que lo justifique.
- **¿Por qué NO React/Vue/Svelte?** El requerimiento prohíbe explícitamente frameworks de UI externos. Además, para un simulador de animaciones Canvas a 60 FPS, el Virtual DOM introduce overhead innecesario. Vanilla JS con requestAnimationFrame es la solución óptima y de menor fricción.
- **¿Por qué NO Vite/Webpack?** No hay transpilación necesaria (ES6+ es soportado nativamente en todos los navegadores modernos). Un bundler agregaría complejidad de toolchain sin beneficio para un artefacto estático de preventa.
- **¿Por qué NO Canvas único?** Se usan 2 canvas superpuestos (particleCanvas + connectionCanvas) tal como especifica el requerimiento, para separar capas de renderizado y optimizar el repintado selectivo. Esta es la única complejidad justificada por la especificación técnica.
- **¿Por qué NO CSS Modules o SASS?** CSS Custom Properties (:root) son suficientes para el sistema de tokens de color requerido. SASS agregaría una dependencia de compilación sin valor incremental.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (Bajo):** Sin bundler, el caché de módulos JS depende del servidor web. Para producción/demo se recomienda servir desde cualquier servidor estático (Live Server, nginx, GitHub Pages). Mitigación: documentar en README.
- **Riesgo 2 (Bajo):** El estado global en memoria se pierde al recargar la página. Aceptable para un simulador de preventa sin requisito de persistencia.

- **Riesgo 3 (Medio):** El loop de requestAnimationFrame con 2 canvas y partículas puede generar consumo de CPU elevado en hardware muy limitado. Mitigación: implementar un cap de partículas configurable en state.js y pausar el loop cuando la pestaña no está visible (document.visibilityState).

5. CONTRATO DE INTERFACES
- No se requieren variables de entorno (aplicación 100% estática).
- Contrato interno JS: El objeto `window.POWER_GRID_STATE` NO debe usarse. Todo acceso al estado debe ser a través de las funciones exportadas de `state.js`.
- Los IDs de DOM críticos que deben respetarse: `#particleCanvas`, `#connectionCanvas`, `#core-nucleus`, `#overdrive-overlay`, `#node-gitlab`, `#node-ci`, `#node-xray`, `#node-datadog`, `#node-rovo`.
- Convención de clases de estado: `.node--active`, `.node--inactive`, `.overdrive-active` en el elemento `<body>`.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 2
- src/state.js | Estado: APROBADO | Iteraciones: 1
- src/kpi-engine.js | Estado: APROBADO | Iteraciones: 1
- src/canvas-renderer.js | Estado: APROBADO | Iteraciones: 2
- src/ui-controller.js | Estado: APROBADO | Iteraciones: 1
- src/main.js | Estado: APROBADO | Iteraciones: 1
- styles.css | Estado: APROBADO | Iteraciones: 2
- README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 105,846
Tokens de salida (output): 36,584
Total tokens: 142,430
Horas-hombre estimadas: ~12.7 horas
Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“2,822 input | !‘1,595 output | 1 llamadas
MICROPLANIFICACION-DISEÑO: !“20,801 input | !‘11,936 output | 8 llamadas
FRONTEND-DESARROLLO: !“30,813 input | !‘21,275 output | 11 llamadas
QA-VALIDACION: !“51,410 input | !‘1,778 output | 11 llamadas